LIFELINE//OPS — Project Brief

An AI emergency dispatcher for 108 (medical) and 1962 (veterinary) in Andhra Pradesh.
A caller phones in — in Telugu, Hindi or English, usually panicking — and the system answers, works out what happened, fills the dispatch card, sends the nearest unit, and scores itself afterwards. Custom-built, not assembled from a platform. Here is why, what is in it, and where it stands.

The stack

Layer	What we use	Why this one
Telephony	Twilio Voice + SIP trunk (`sip.voice.x.ai`), ngrok for dev	Only carrier path that already works for Indian numbers with webhook control
Backend	Python 3.11, FastAPI, single process on `:5000` , 43 endpoints	One file to run, no orchestrator to babysit
STT	Groq Whisper · Sarvam · Gemini (swappable per call)	Sarvam is the one that actually hears Telugu; the others are fallbacks
LLM	Groq Llama 3.3 · Gemini · Claude · Grok · Ollama (local)	Provider is a setting, not a rewrite — needed for A/B and for key outages
TTS	Sarvam · Groq PlayAI · Gemini	Same reason: Indian-language voice quality is the deciding factor
Noise	**Hush** speech-enhancement sidecar (`external/Hush` , own venv, port 8600)	8 MB CPU model; runs out-of-process so torch never enters the backend venv, and degrades to raw audio if it is down
Retrieval	MongoDB `lifeline_kb` + hybrid RAG (dense multilingual embeddings + BM25 → RRF → cross-encoder rerank)	Cross-lingual retrieval; a Telugu query has to find an English-tagged case
Storage	SQLite, one file `database/108-1962.sql` , 11 tables	The whole call record is one file you can copy and hand over
Console	`frontend/bavya.html` — single-file, no build step	Human Console + Audit Log, editable by anyone, deployable by copying
Tests	pytest (141 tests) + `node` for browser logic, GitHub Actions on every push	See below — this is the part that matters
Evals	promptfoo suites under `evals/`	Accuracy claims have to be measured, not asserted

Layer	What we use	Why this one
Simulation	`sim/simulate.py` — LLM plays the caller, real agent answers, batch of scenarios	Testing by making phone calls one at a time does not scale

---

What we built

The call path. Twilio webhook → STT → LLM with five tools (`dispatch_ambulance` , `classify_emergency` , `lookup_protocol` , `escalate_to_human` , `store_caller_info`) → TTS → SQLite. Per-caller memory, a first-aid knowledge base, and a tool-call log for every invocation.

The dispatch card (`backend/call_state.py`). A human operator works from a form. So does this agent. The model decides what to say and in which language; the module decides only whether a required field is still empty. No dialogue tree, no keyword matching — just a checklist and two gates reading it. Multi-victim and bystander cases have explicit waivers, because a stranger on the footpath has no name to give.

Deterministic dispatch (`backend/dispatch.py`). Unit selection, distance, ETA and fleet reservation were pulled out of the LLM's reach entirely. It replaced two copy-pasted implementations that had drifted apart, and killed a `random.randint(6,12)` ETA. A base with no coordinate is never offered as "nearby" — the call escalates to a human instead of getting a confident invented number.

Honest QA scoring (`backend/qa_engine.py`). The old scorer counted English words ("okay", "got it", "sir"). It rated a 394 km wrong-city dispatch about the same as a perfect call, and rated a flawless Telugu call near zero. The rubric is now built from facts the system already knows — which unit went, how far, was the patient identified, did the call close properly. The two genuinely language-dependent parameters are judged by a model and reported as *not assessed* when no provider is available, never guessed.

Retrieval few-shot (`rag/` + `backend/successful_calls_rag.py`). QA-approved past calls are retrieved by embedding similarity on the partial transcript and distilled into a per-type playbook — question order, must-collect fields, tone — instead of dumping ten raw transcripts into the prompt. It degrades to an empty string on any failure, so a live call never breaks because Mongo is down.

A test suite that is a list of real failures. Every file corresponds to something that reached a live 108 call: ambulances chosen by table order, Telugu place names failing to match a base, "brother" accepted as a patient's name, a bystander refused a vehicle, a

waiver claimed without asking, a model inventing an argument name and killing the turn with an HTTP 500, the browser waiving name and age on turn one.

Where it stands

- **141 tests, all passing**, in about five seconds. No keys, no network, no phone, and never touching the real database. Green on every push via GitHub Actions.
- **Triage eval baseline: 17/19 service accuracy, 19/19 severity.** The two failures are the headroom, and we know exactly what they are.
- **Extractor eval: 5/6** — the one failure is a real bug the harness caught before a caller did: the model extracted a field and then still listed it as missing, which would stall dispatch in the live app.
- **RAG retriever: 6/6**, including the headline cross-lingual case — a Telugu cow-dystocia query retrieves the right 1962 case ranked #1.
- **Simulator built and running** against the real agent and the real gates. Batch runs are not green yet; the last full pass was mostly infrastructure errors, not agent errors, and it is the current work.
- **Known gap, flagged not hidden:** `classify_emergency` is still pure keyword matching with an English-only word list — the exact brittleness behind the Telugu misroute. The simulator also re-implements the turn sequence rather than sharing it with the console; the fix is moving orchestration to the backend.

Why not Dograh

Dograh is a capable open-source voice-AI agent platform, and on paper it covers a lot of this: telephony, STT/LLM/TTS orchestration, a workflow builder, call logging. We looked at it seriously. We did not use it, for one reason:

It gives a great many things, and not the ones we need.

The parts we actually spend our engineering on are the parts a general platform treats as your business logic and leaves to you anyway:

- a dispatch card with *this* service's required fields and *this* service's waiver rules
- unit selection against the real AP ambulance roster, with Telugu place-name matching and an ETA that refuses to be invented
- a QA rubric that scores outcomes rather than English words
- a knowledge base of QA-approved 108/1962 calls, retrieved cross-lingually
- gates that decide whether a vehicle is allowed to move

To get those inside Dograh we would be modifying its code. It is open source, so that is permitted — but permitted is not the same as cheap. Every change means reading someone else's architecture, changing it, testing it against behaviour we do not control, and then carrying that fork forward through every upstream release. That effort produces no dispatch accuracy. It produces a fork.

And the honest part: we could not establish, up front, that it would bend to these requirements at all. Emergency dispatch is protocol-driven and locale-specific. Betting a project on "we can probably customise it" is a bet you only discover you lost late, after the integration work is already sunk.

So we went custom. The trade is real and we accept it — we own the telephony plumbing, the provider adapters and the console, which a platform would have handed us. What we get back is that every line in this repo is one we needed, the gates that decide whether an ambulance moves are ours and are tested like arithmetic, and when a call goes wrong we can find out why in five seconds instead of filing an upstream issue.

The old prototype still exists on the `dograh` branch if it is ever needed again. Nothing was thrown away.